

09C19 도전과제 — 중위수 필터

코드 설명 (2페이지) · 09_challenge_1.ino

1. 과제 조건

- 09_example_1을 재사용하되, 범위 밖일 때 직전 유효값을 쓰는 코드는 제거했다.
- timeout(0) 등 오류 측정값이 버퍼에 그대로 들어가게 해서, 중위수 필터만으로 노이즈가 제거되는지 확인한다.
- 최근 N개 샘플을 유지하는 자료구조와, 그 N개 중 중위수를 반환하는 함수를 구현했다.

2. N개 샘플 자료구조

고정 배열 원형 버퍼 samples[30]에 최신 측정값을 넣는다. add_sample()이 인덱스를 한 칸씩 돌리며 가장 오래된 값을 덮어쓴다. 버퍼가 가득 찰 때까지는 sample_count개만 정렬한다.

```
const int MEDIAN_NS[] = {3, 10, 30};
#define N_SELECT 1 // 1 → N=3, 2 → N=10, 3 → N=30
```

setup()에서 n_samples = MEDIAN_NS[N_SELECT - 1]로 창 크기를 고른다. 시작 시 LED가 1/2/3번 깜빡여 어떤 N으로 올렸는지 확인한다.

3. 중위수 필터

median_filter()는 원형 버퍼를 복사한 뒤 삽입 정렬한다. 원본 순서는 유지된다.

- N이 홀수(3, 30): 가운데 값 buf[n/2]
- N이 짝수(10): 가운데 두 값의 평균 (buf[4] + buf[5]) / 2

timeout으로 0이 한두 번 섞여도 정렬 후 양 끝으로 밀리므로, 중위수는 정상 거리 근처에 남는다. 이것이 직전 유효값 대체 없이 스파이크를 막는 이유이다.

4. EMA (비교용)

슬라이드 출력 형식에 ema가 있어, 범위 필터 없이 raw에 EMA를 걸었다.

```
dist_ema =  $\alpha$  · dist_raw + (1 -  $\alpha$ ) · dist_ema    ( $\alpha = 0.5$ )
```

EMA는 0에도 끌리므로 raw 스파이크가 남는다. 같은 플로터에서 median이 timeout을 더 잘 막는 것을 비교할 수 있다.

5. 시리얼 플로터 출력

슬라이드 20쪽 형식을 그대로 사용한다.

```
Serial.print("Min:"); Serial.print(_DIST_MIN);  
Serial.print(",raw:"); Serial.print(dist_raw);  
Serial.print(",ema:"); Serial.print(dist_ema);  
Serial.print(",median:"); Serial.print(dist_median);  
Serial.print(",Max:"); Serial.print(_DIST_MAX);  
Serial.println("");
```

N=3, 10, 30 스크린샷은 N_SELECT를 1→2→3으로 바꿔 업로드한 뒤 57600 보드레이트로 찍는다.

6. N에 따른 차이

N	장점	단점
3	움직임 반응이 빠르다	오류가 연속이면 덜 걸러진다
10	노이즈 제거와 반응의 균형	짝수라 가운데 두 값을 평균한다
30	timeout 제거가 가장 강하다	거리 변화가 늦게 보인다